Step 2: Software Development Project

Week 2 – Core Functionality Implementation

1. Selected Functional Requirements

For Week 2, I selected four functional requirements from my original requirements document. These requirements focus on processing research papers and retrieving relevant information.

- **FR-04:** The system shall divide extracted text into smaller chunks for retrieval.
- **FR-05:** The system shall generate embeddings for document chunks.
- **FR-06:** The system shall store embeddings and associated metadata in a vector store.
- **FR-08:** The system shall retrieve document chunks relevant to a question.

2. Functionality Implemented

FR-04 – Text Chunking

After extracting text from a PDF, the system divides the text into smaller chunks using LangChain's RecursiveCharacterTextSplitter.

The current settings are: Chunk size: 1,000 characters, Chunk overlap: 200 characters

Each chunk also keeps its original PDF page number. This helps the application identify where the retrieved information came from.

Source module: `src/ingestion/text_splitter.py`

FR-05 – Embedding Generation

The system uses the local Ollama embedding model `nomic-embed-text` to generate embeddings for document chunks. The embedding model converts text into numerical vectors that can be used to find information with similar meaning.

Source module: `src/ingestion/embedding.py`

FR-06 – Vector Storage

The application uses ChromaDB to store document chunks and their associated metadata.

The stored metadata includes: Original PDF page number, PDF source ID

The application creates a unique source ID from the uploaded PDF's content. It also creates predictable IDs for individual chunks. Before storing a processed PDF again,

the application removes its previously stored chunks for that PDF. This prevents duplicate records when the same document is processed multiple times.

The vector store is saved locally in `data/chroma/`

Source module: `src/retrieval/vector_store.py`

FR-08 – Information Retrieval

Users can enter a question about an uploaded research paper. The application uses ChromaDB similarity to search to find the most relevant stored passages.The search currently retrieves the top three results by default and filters them using the selected PDF's source ID. This helps prevent results from other uploaded papers from appearing in the search.

The application displays each retrieved passage along with its PDF page number.

Source modules: `src/retrieval/vector_store.py` and `src/app.py`

3. Component Interaction

The main components now work together in the following order:

1. The user uploads an academic research PDF through the Streamlit interface.
2. The PDF loader extracts text from each page.
3. The text splitter splits extracted text into smaller chunks and keeps page numbers.
4. The embedding model generates embeddings for the chunks.
5. ChromaDB stores chunks, embeddings, and metadata.
6. The user enters a question about the uploaded paper.
7. The retrieval component searches for relevant chunks using similarity search.
8. The application displays the retrieved passages and their page numbers.

This integration allows the application to process research papers and retrieve relevant information based on user questions.

4. Testing Summary and Results

Automated tests were created using pytest. The tests check the main functionality of the PDF loader, text splitter, embedding model, and vector store.

Automated test result: 9 tests passed

Test file	Tests	Result	Test Coverage
test_pdf_loader.py	2	Passed	PDF page extraction and handling of empty pages
test_text_splitter.py	2	Passed	Text chunk creation and page-number preservation
test_embedding.py	1	Passed	Embedding model configuration
test_vector_store.py	4	Passed	Vector store creation, chunk storage, metadata and chunk IDs, duplicate prevention, similarity search with source-specific filtering, and empty chunk handling
Total	9	All passed	

4.1 Manual Testing

I also tested the application using three academic research PDFs.

The tests confirmed that:

- PDF text was extracted and divided into chunks.
- The application stored the chunks in ChromaDB.
- Questions returned relevant passages from the uploaded paper.
- Retrieved results displayed their original page numbers.
- Searching for one PDF did not return chunks belonging to another PDF.
- Processing the same PDF again did not create duplicate stored chunks.

For example, I tested a question about the two main areas of human responsibility under the compressed software development life cycle. The application retrieved relevant passages from page 2 of the research paper.

The results show that the core retrieval functionality is working. However, the relevance of search results can vary depending on how a question is written.

To run the automated tests, I used: `uv run pytest -v`

5. Significant Technical Problems

During implementation, I encountered and fixed several issues.

1. Duplicate chunks when processing the same PDF

Step 2: Software Development Project

Initially, processing the same PDF more than once created duplicate records in ChromaDB. I fixed this by deleting existing records for the same PDF source ID before adding the new chunks.

2. Import and module organization issues

I encountered an import issue while organizing the PDF loader and other project modules. I corrected the imports and module structure so that the application and automated tests could access the required functions.

3. Search result relevance

Some questions returned passages that were only partially related to the question. I tested different question wordings and found that clearer, more specific questions often returned better results.

4. PDF text extraction limitations

Some PDF text may contain unusual spacing or words split across lines. This can affect the quality of the extracted text and search results.

6. Incomplete Functionality and Further Work

The current implementation focuses on document processing, embedding generation, vector storage, and retrieval. Some planned features are not yet complete.

The following work remains:

- LLM answer generation: Integrate the local Llama 3.2 model so the application can generate answers using retrieved passages.
- Source references in answers: Display supporting PDF page references with generated answers.
- Retrieval improvement: Continue testing and improving the relevance of retrieved passages.
- Scanned PDF support: Investigate OCR support for PDFs that contain scanned images instead of selectable text.
- Error handling: Continue testing invalid files, empty documents, and other error conditions.

The next development stage will focus on connecting the retrieved passages to the LLM so that users can receive answers based on the uploaded research paper.